this is the link from where we got the dataset : https://www.kaggle.com/datasets/miadul/irrigation-water-requirement-prediction-dataset

all informations about this dataset are :

- number of observations : 10 000
- number of features : 20

In [1]:
```
#l'importation des bibliotheques necessaires
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
#la definition de note dataframe
dataset_path = "irrigation_prediction.csv"
df = pd.read_csv(dataset_path)
```

In [3]:
```
df.head()
```

Out[3]:

	Soil_Type	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity
0	Clay	6.14	36.48	0.42	2.17	21.90	31.19
1	Silt	6.41	50.56	0.38	0.23	36.50	26.01
2	Sandy	7.71	40.07	1.09	2.18	41.83	76.41
3	Clay	5.96	12.75	1.56	0.40	37.22	43.32
4	Clay	7.76	18.58	0.95	2.52	22.38	86.44

In [4]:
```
df.describe()
#on peut remarquer que pour la plupart des variables ont des valeur logique en se basant sur
```

Out[4]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humid
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.0000
mean	6.487857	36.969207	0.944731	1.791963	26.991423	60.0803
std	0.979963	16.430845	0.372406	0.984202	8.664074	20.1879
min	4.800000	8.000000	0.300000	0.100000	12.000000	25.0000
25%	5.640000	22.860000	0.620000	0.940000	19.460000	42.8550
50%	6.470000	37.240000	0.950000	1.780000	27.090000	60.0400
75%	7.350000	50.940000	1.260000	2.650000	34.500000	77.7050
max	8.200000	65.000000	1.600000	3.500000	42.000000	95.0000

In [5]:
```
df.info()
#on peut remarquer qu'il y a aucune valeur manquantes,et que notre dataset a deux types de do
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 20 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Soil_Type                10000 non-null  object
 1   Soil_pH                  10000 non-null  float64
 2   Soil_Moisture            10000 non-null  float64
 3   Organic_Carbon           10000 non-null  float64
 4   Electrical_Conductivity  10000 non-null  float64
 5   Temperature_C            10000 non-null  float64
 6   Humidity                 10000 non-null  float64
 7   Rainfall_mm              10000 non-null  float64
 8   Sunlight_Hours           10000 non-null  float64
 9   Wind_Speed_kmh           10000 non-null  float64
 10  Crop_Type                10000 non-null  object
 11  Crop_Growth_Stage        10000 non-null  object
 12  Season                   10000 non-null  object
 13  Irrigation_Type          10000 non-null  object
 14  Water_Source             10000 non-null  object
 15  Field_Area_hectare       10000 non-null  float64
 16  Mulching_Used            10000 non-null  object
 17  Previous_Irrigation_mm   10000 non-null  float64
 18  Region                   10000 non-null  object
 19  Irrigation_Need          10000 non-null  object
dtypes: float64(11), object(9)
memory usage: 1.5+ MB
```

In [6]:
```
#avant tout, on fait la suppression des observations doublantes
df = df.drop_duplicates()
```

In [7]:
```
df.info()
#alors on n'a pas des doublantes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 20 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Soil_Type                10000 non-null  object
 1   Soil_pH                  10000 non-null  float64
 2   Soil_Moisture            10000 non-null  float64
 3   Organic_Carbon           10000 non-null  float64
 4   Electrical_Conductivity  10000 non-null  float64
 5   Temperature_C            10000 non-null  float64
 6   Humidity                 10000 non-null  float64
 7   Rainfall_mm              10000 non-null  float64
 8   Sunlight_Hours           10000 non-null  float64
 9   Wind_Speed_kmh           10000 non-null  float64
 10  Crop_Type                10000 non-null  object
 11  Crop_Growth_Stage        10000 non-null  object
 12  Season                   10000 non-null  object
 13  Irrigation_Type          10000 non-null  object
 14  Water_Source             10000 non-null  object
 15  Field_Area_hectare       10000 non-null  float64
 16  Mulching_Used            10000 non-null  object
 17  Previous_Irrigation_mm   10000 non-null  float64
 18  Region                   10000 non-null  object
 19  Irrigation_Need          10000 non-null  object
dtypes: float64(11), object(9)
memory usage: 1.5+ MB
```

In [8]:
```
#on fait la separation des variables categorielles et numeriques pour faire le pre-traitement
x_num = df.select_dtypes(include = [np.number])
x_str = df.select_dtypes(exclude = [np.number])
```

In [9]:
```
x_num.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Soil_pH                  10000 non-null  float64
 1   Soil_Moisture            10000 non-null  float64
 2   Organic_Carbon           10000 non-null  float64
 3   Electrical_Conductivity  10000 non-null  float64
 4   Temperature_C            10000 non-null  float64
 5   Humidity                 10000 non-null  float64
 6   Rainfall_mm              10000 non-null  float64
 7   Sunlight_Hours           10000 non-null  float64
 8   Wind_Speed_kmh           10000 non-null  float64
 9   Field_Area_hectare       10000 non-null  float64
 10  Previous_Irrigation_mm   10000 non-null  float64
dtypes: float64(11)
memory usage: 859.5 KB
```

In [10]:
```
x_str.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Soil_Type          10000 non-null  object
 1   Crop_Type          10000 non-null  object
 2   Crop_Growth_Stage  10000 non-null  object
 3   Season             10000 non-null  object
 4   Irrigation_Type    10000 non-null  object
 5   Water_Source       10000 non-null  object
 6   Mulching_Used      10000 non-null  object
 7   Region             10000 non-null  object
 8   Irrigation_Need    10000 non-null  object
dtypes: object(9)
memory usage: 703.2+ KB
```

In [11]:
```
#meme si on n'a pas des valeurs manquantes mais on va proceder au remplissage juste pour des r
#on commence par les variables numeriques
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

imputer = IterativeImputer(random_state = 43)

x_num_df = imputer.fit_transform(x_num)
imputed_x_num = pd.DataFrame(x_num_df, columns = x_num.columns)
```

In [12]:
```
imputed_x_num.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Soil_pH                  10000 non-null  float64
 1   Soil_Moisture            10000 non-null  float64
 2   Organic_Carbon           10000 non-null  float64
 3   Electrical_Conductivity  10000 non-null  float64
 4   Temperature_C            10000 non-null  float64
 5   Humidity                 10000 non-null  float64
 6   Rainfall_mm              10000 non-null  float64
 7   Sunlight_Hours           10000 non-null  float64
 8   Wind_Speed_kmh           10000 non-null  float64
 9   Field_Area_hectare       10000 non-null  float64
 10  Previous_Irrigation_mm   10000 non-null  float64
dtypes: float64(11)
memory usage: 859.5 KB
```

In [13]:

```
#puis on passe au traitement des outliers
def outliers(col):
    Q1 = col.quantile(0.25)
    Q3 = col.quantile(0.75)

    IQR = Q3 - Q1

    Fb = Q1 - 1.5 * IQR
    Fh = Q3 + 1.5 * IQR

    return col.clip(lower = Fb, upper = Fh)

for col in imputed_x_num.columns :
    imputed_x_num[col] = outliers(imputed_x_num[col])
```

In [14]:

```
imputed_x_num.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Soil_pH                  10000 non-null  float64
 1   Soil_Moisture            10000 non-null  float64
 2   Organic_Carbon           10000 non-null  float64
 3   Electrical_Conductivity  10000 non-null  float64
 4   Temperature_C            10000 non-null  float64
 5   Humidity                 10000 non-null  float64
 6   Rainfall_mm              10000 non-null  float64
 7   Sunlight_Hours           10000 non-null  float64
 8   Wind_Speed_kmh           10000 non-null  float64
 9   Field_Area_hectare       10000 non-null  float64
 10  Previous_Irrigation_mm   10000 non-null  float64
dtypes: float64(11)
memory usage: 859.5 KB
```

In [15]:

```
#on commence par l'imputation
for col in x_str.columns:
    x_str[col] = x_str[col].fillna(x_str[col].mode()[0])
```

In [16]:

```
x_str.describe()
```

Out[16]:

	Soil_Type	Crop_Type	Crop_Growth_Stage	Season	Irrigation_Type	Water_Source	Mulching_U
count	10000	10000	10000	10000	10000	10000	10
unique	4	6	4	3	4	4	
top	Sandy	Rice	Harvest	Rabi	Sprinkler	River	
freq	2536	1711	2581	3383	2527	2528	5

In [17]:
```
y = x_str["Irrigation_Need"]
x_str = x_str.drop(columns = "Irrigation_Need")
```

In [18]:
```
#maintenant on fait la concatination des str et num
x = pd.concat([x_str,imputed_x_num], axis = 1)
```

In [19]:
```
#on fait la divsion des donnees en donnees d'entrainement et de test
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x,y, random_state = 43, test_size = 0.2)
```

In [20]:
```
x_train
```

Out[20]:

	Soil_Type	Crop_Type	Crop_Growth_Stage	Season	Irrigation_Type	Water_Source	Mulching_Use
6100	Loamy	Potato	Sowing	Kharif	Sprinkler	River	N
4860	Silt	Cotton	Harvest	Rabi	Drip	Reservoir	Ye
4401	Clay	Sugarcane	Vegetative	Kharif	Sprinkler	Reservoir	N
1416	Sandy	Rice	Sowing	Zaid	Canal	Rainwater	N
8620	Sandy	Maize	Flowering	Zaid	Drip	Reservoir	Ye
...	...	...	...	...	...	...	
8499	Sandy	Cotton	Vegetative	Rabi	Sprinkler	Rainwater	Ye
2064	Clay	Potato	Flowering	Kharif	Sprinkler	Reservoir	Ye
7985	Clay	Cotton	Vegetative	Zaid	Sprinkler	Rainwater	N
2303	Sandy	Wheat	Sowing	Zaid	Canal	Groundwater	Ye
3392	Loamy	Cotton	Flowering	Zaid	Drip	Groundwater	N

8000 rows × 19 columns

In [21]:
```
x_num_train = x_train.select_dtypes(include = [np.number])
x_str_train = x_train.select_dtypes(exclude = [np.number])


#pour le test
x_num_test = x_test.select_dtypes(include = [np.number])
x_str_test = x_test.select_dtypes(exclude = [np.number])
```

In [22]:
```
x_str_train
```

Out[22]:

	Soil_Type	Crop_Type	Crop_Growth_Stage	Season	Irrigation_Type	Water_Source	Mulching_Use
6100	Loamy	Potato	Sowing	Kharif	Sprinkler	River	N
4860	Silt	Cotton	Harvest	Rabi	Drip	Reservoir	Ye
4401	Clay	Sugarcane	Vegetative	Kharif	Sprinkler	Reservoir	N
1416	Sandy	Rice	Sowing	Zaid	Canal	Rainwater	N
8620	Sandy	Maize	Flowering	Zaid	Drip	Reservoir	Ye
...	...	...	...	...	...	...	
8499	Sandy	Cotton	Vegetative	Rabi	Sprinkler	Rainwater	Ye
2064	Clay	Potato	Flowering	Kharif	Sprinkler	Reservoir	Ye
7985	Clay	Cotton	Vegetative	Zaid	Sprinkler	Rainwater	N
2303	Sandy	Wheat	Sowing	Zaid	Canal	Groundwater	Ye
3392	Loamy	Cotton	Flowering	Zaid	Drip	Groundwater	N

8000 rows × 8 columns

In [23]:

```
x_num_train
```

Out[23]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Rainf
6100	5.73	47.15	0.75	3.05	31.70	55.52	
4860	7.69	49.82	0.72	1.60	24.01	92.93	
4401	5.80	36.50	0.96	2.15	19.79	43.00	
1416	6.11	63.12	0.43	0.55	30.48	60.28	
8620	6.44	55.00	0.56	2.04	35.65	42.78	
...	...	...	...	...	...	...	
8499	7.21	35.18	0.33	0.58	31.77	32.20	
2064	5.25	54.31	0.69	2.96	35.72	54.86	
7985	7.89	53.39	1.09	0.18	16.36	29.47	
2303	7.82	13.12	0.91	1.54	41.77	46.23	
3392	8.06	20.76	1.44	0.15	28.45	71.52	

8000 rows × 11 columns

In [ ]:

In [ ]:

In [24]:

```
#on fait maintenant la standarisation
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

x_train_s = scaler.fit_transform(x_num_train)
```

```
#pour le test
x_test_s = scaler.transform(x_num_test)
```

In [25]:
```
x_train_s = pd.DataFrame(x_train_s, columns = imputed_x_num.columns)



#pour le test
x_test_s = pd.DataFrame(x_test_s, columns = imputed_x_num.columns)
```

In [26]:
```
x_train_s
```

Out[26]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Rai
0	-0.772932	0.627754	-0.527456	1.274816	0.524863	-0.234736	-
1	1.218379	0.790359	-0.608031	-0.198392	-0.363072	1.617483	-
2	-0.701814	-0.020841	0.036568	0.360411	-0.850340	-0.854617	
3	-0.386862	1.600341	-1.386920	-1.265197	0.383994	0.000938	-
4	-0.051590	1.105826	-1.037763	0.248651	0.980955	-0.865510	-
...	...	...	...	...	...	...	
7995	0.730711	-0.101230	-1.655502	-1.234717	0.532946	-1.389340	-
7996	-1.260600	1.063804	-0.688605	1.183376	0.989038	-0.267413	
7997	1.421574	1.007775	0.385725	-1.641119	-1.246390	-1.524506	
7998	1.350456	-1.444704	-0.097724	-0.259352	1.687609	-0.694696	-
7999	1.594290	-0.979421	1.325764	-1.671599	0.149598	0.557445	-

8000 rows × 11 columns

In [27]:
```
#et donc avant de split les donnees en fait l'encodage
from sklearn.preprocessing import OneHotEncoder, LabelEncoder

encoder = OneHotEncoder(sparse_output = False, handle_unknown = 'ignore')

encoded_x_str_train = encoder.fit_transform(x_str_train)
encoded_x_str_df = pd.DataFrame(encoded_x_str_train, columns = encoder.get_feature_names_out(:




#pour le test
encoded_x_str_test = encoder.transform(x_str_test)
encoded_x_str_df_test = pd.DataFrame(encoded_x_str_test, columns = encoder.get_feature_names_
```

In [28]:
```
encoded_x_str_df
```

Out[28]:

	Soil_Type_Clay	Soil_Type_Loamy	Soil_Type_Sandy	Soil_Type_Silt	Crop_Type_Cotton	Crop_Type_M
0	0.0	1.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	1.0	1.0	
2	1.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	1.0	0.0	0.0	
4	0.0	0.0	1.0	0.0	0.0	
...	...	...	...	...	...	
7995	0.0	0.0	1.0	0.0	1.0	
7996	1.0	0.0	0.0	0.0	0.0	
7997	1.0	0.0	0.0	0.0	1.0	
7998	0.0	0.0	1.0	0.0	0.0	
7999	0.0	1.0	0.0	0.0	1.0	

8000 rows × 32 columns

In [29]:

```
#maintenant on fait la concatination des str et num
x_train = pd.concat([x_train_s,encoded_x_str_df], axis = 1)



#pour le test
x_test = pd.concat([x_test_s,encoded_x_str_df_test], axis = 1)
```

In [30]:

```
x_train
```

Out[30]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Rai
0	-0.772932	0.627754	-0.527456	1.274816	0.524863	-0.234736	-
1	1.218379	0.790359	-0.608031	-0.198392	-0.363072	1.617483	-
2	-0.701814	-0.020841	0.036568	0.360411	-0.850340	-0.854617	
3	-0.386862	1.600341	-1.386920	-1.265197	0.383994	0.000938	-
4	-0.051590	1.105826	-1.037763	0.248651	0.980955	-0.865510	-
...	...	...	...	...	...	...	
7995	0.730711	-0.101230	-1.655502	-1.234717	0.532946	-1.389340	-
7996	-1.260600	1.063804	-0.688605	1.183376	0.989038	-0.267413	
7997	1.421574	1.007775	0.385725	-1.641119	-1.246390	-1.524506	
7998	1.350456	-1.444704	-0.097724	-0.259352	1.687609	-0.694696	-
7999	1.594290	-0.979421	1.325764	-1.671599	0.149598	0.557445	-

8000 rows × 43 columns

In [31]:

```
#on fait l'encodage de y par labelencoder
l_encoder = LabelEncoder()
```

```
encoded_y = l_encoder.fit_transform(y_train)
encoded_y_df = pd.DataFrame(encoded_y)



#pour le test
encoded_y_test = l_encoder.transform(y_test)
encoded_y_df_test = pd.DataFrame(encoded_y_test)
encoded_y_df_test.columns = ["Irrigation_Need"]
```

In [32]:
```
encoded_y_df.columns = ["Irrigation_Need"]
```

In [33]:
```
encoded_y_df
```

Out[33]:

	Irrigation_Need
0	1
1	1
2	2
3	2
4	1
...	...
7995	2
7996	2
7997	2
7998	1
7999	2

8000 rows × 1 columns

In [34]:
```
y_train = encoded_y_df


#pour le test
y_test = encoded_y_df_test
```

In [35]:
```
df_temp = pd.concat([x_train,y_train], axis = 1)
```

In [36]:
```
df_temp
```

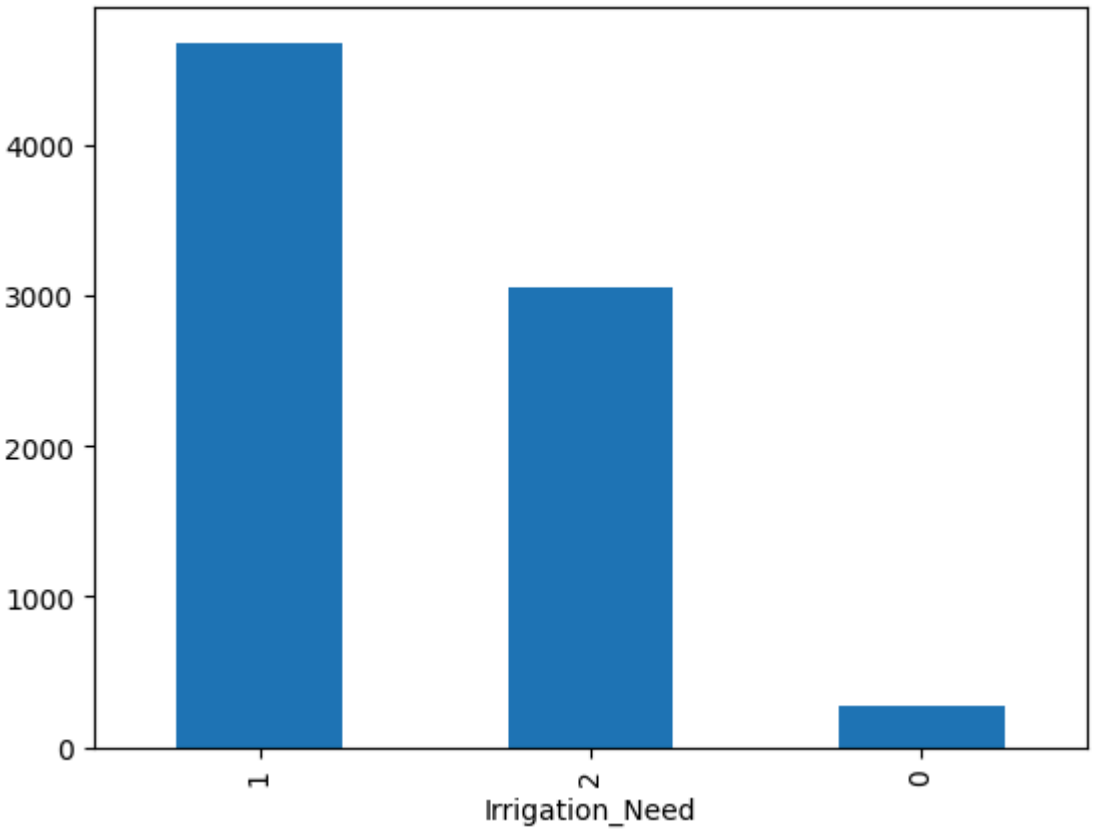
Out[36]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Rai
0	-0.772932	0.627754	-0.527456	1.274816	0.524863	-0.234736	-
1	1.218379	0.790359	-0.608031	-0.198392	-0.363072	1.617483	-
2	-0.701814	-0.020841	0.036568	0.360411	-0.850340	-0.854617	
3	-0.386862	1.600341	-1.386920	-1.265197	0.383994	0.000938	-
4	-0.051590	1.105826	-1.037763	0.248651	0.980955	-0.865510	-
...	...	...	...	...	...	...	
7995	0.730711	-0.101230	-1.655502	-1.234717	0.532946	-1.389340	-
7996	-1.260600	1.063804	-0.688605	1.183376	0.989038	-0.267413	
7997	1.421574	1.007775	0.385725	-1.641119	-1.246390	-1.524506	
7998	1.350456	-1.444704	-0.097724	-0.259352	1.687609	-0.694696	-
7999	1.594290	-0.979421	1.325764	-1.671599	0.149598	0.557445	-

8000 rows × 44 columns

In [37]:
```
df_temp["Irrigation_Need"].value_counts().plot(kind = 'bar')
#on peut voir que le dataset n'est pas equilibrée (low tres domainant)
```

Out[37]:
```
<Axes: xlabel='Irrigation_Need'>
```



In [38]:
```
#on va utilier la methode de SMOTE qui sert a cree des nouvelles observations reelles
```

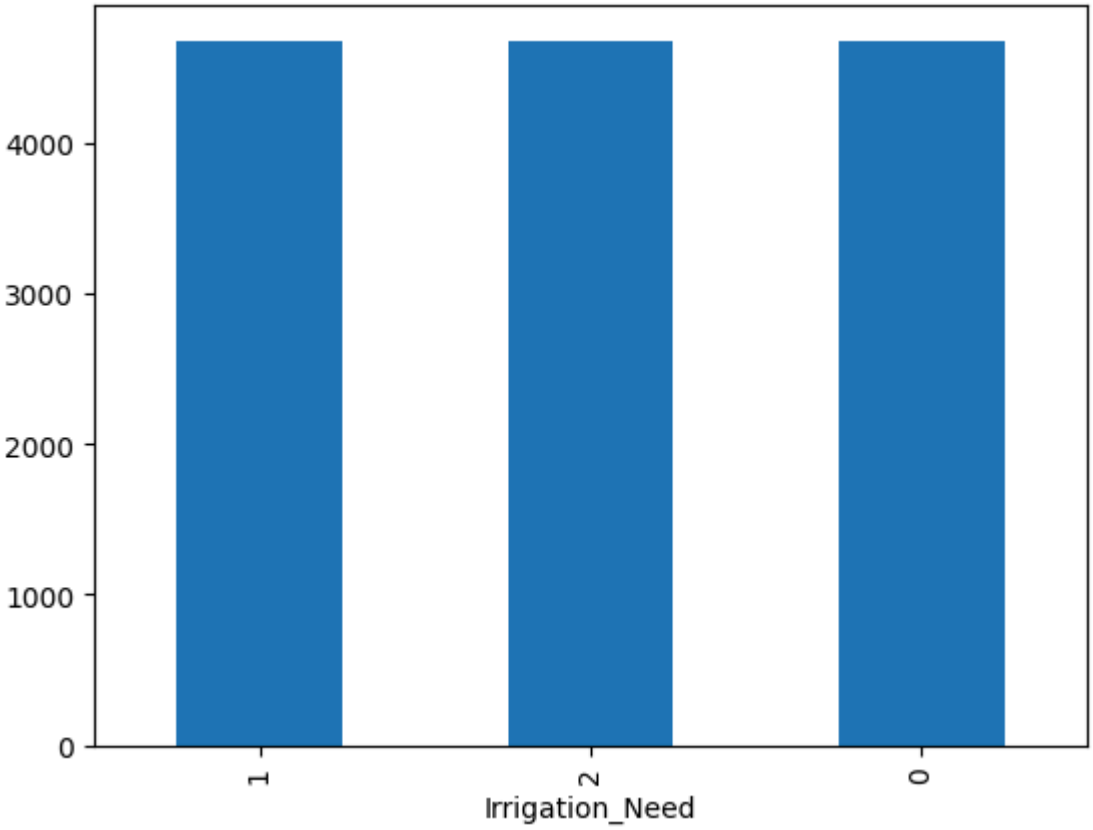
In [39]:
```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state = 43)

x_train_equ, y_train_equ = smote.fit_resample(x_train, y_train)
```

In [40]:
```
y_train_equ['Irrigation_Need'].value_counts().plot(kind="bar")
#alors on voit que notre dataset est bien equilibré maintenant
```

Out[40]: <Axes: xlabel='Irrigation_Need'>



In [41]:
```
x_train_equ
```

Out[41]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Ra
0	-0.772932	0.627754	-0.527456	1.274816	0.524863	-0.234736	
1	1.218379	0.790359	-0.608031	-0.198392	-0.363072	1.617483	
2	-0.701814	-0.020841	0.036568	0.360411	-0.850340	-0.854617	
3	-0.386862	1.600341	-1.386920	-1.265197	0.383994	0.000938	
4	-0.051590	1.105826	-1.037763	0.248651	0.980955	-0.865510	
...	...	...	...	...	...	...	
14020	1.597521	-0.624620	-0.873648	0.555655	0.949054	1.435202	
14021	0.116458	1.507710	-0.417327	0.708109	0.417210	-0.862673	
14022	-1.097681	0.861795	0.420896	0.726830	1.383996	-1.346348	
14023	-0.336063	-1.280643	1.416162	1.074260	1.215809	0.828376	
14024	0.980499	1.405185	-0.279019	-0.412891	1.387000	-0.374898	

14025 rows × 43 columns

In [42]:
```
y_train_equ
```

Out[42]:

	Irrigation_Need
0	1
1	1
2	2
3	2
4	1
...	...
14020	2
14021	2
14022	2
14023	2
14024	2

14025 rows × 1 columns

In [43]:
```
x_train
```

Out[43]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Rai
0	-0.772932	0.627754	-0.527456	1.274816	0.524863	-0.234736	-
1	1.218379	0.790359	-0.608031	-0.198392	-0.363072	1.617483	-
2	-0.701814	-0.020841	0.036568	0.360411	-0.850340	-0.854617	
3	-0.386862	1.600341	-1.386920	-1.265197	0.383994	0.000938	-
4	-0.051590	1.105826	-1.037763	0.248651	0.980955	-0.865510	-
...	...	...	...	...	...	...	
7995	0.730711	-0.101230	-1.655502	-1.234717	0.532946	-1.389340	-
7996	-1.260600	1.063804	-0.688605	1.183376	0.989038	-0.267413	
7997	1.421574	1.007775	0.385725	-1.641119	-1.246390	-1.524506	
7998	1.350456	-1.444704	-0.097724	-0.259352	1.687609	-0.694696	-
7999	1.594290	-0.979421	1.325764	-1.671599	0.149598	0.557445	-

8000 rows × 43 columns

In [44]:
```
x_test
```

Out[44]:

	Soil_pH	Soil_Moisture	Organic_Carbon	Electrical_Conductivity	Temperature_C	Humidity	Rai
0	0.456398	-0.798546	-0.446881	-1.356638	-0.275318	-1.010578	-
1	-0.102389	0.830554	-1.574928	0.451852	-0.761431	0.096000	
2	1.655248	0.503516	-0.178298	-1.681759	-0.374619	1.569952	-
3	0.547835	1.669159	1.218331	-1.061996	0.277765	-0.965523	
4	0.253203	-1.082953	-1.440636	0.045450	-1.671306	-1.148715	-
...	...	...	...	...	...	...	
1995	1.695887	0.761127	-1.333203	-1.051836	-1.210595	-0.925419	-
1996	1.248858	0.560154	0.976606	1.721858	0.524863	-0.647165	-
1997	-0.965968	0.102179	-1.709219	1.284976	0.064153	-0.480312	
1998	-0.569737	-1.754690	1.433197	1.732019	-0.679450	-1.397757	-
1999	0.944066	1.221538	1.245189	-0.086631	1.269621	-0.194136	-

2000 rows × 43 columns

In [45]:
```
y_train
```

Out[45]:

	Irrigation_Need
0	1
1	1
2	2
3	2
4	1
...	...
7995	2
7996	2
7997	2
7998	1
7999	2

8000 rows × 1 columns

In [46]:
```
y_test
```

Out[46]:

	Irrigation_Need
0	1
1	1
2	2
3	1
4	0
...	...
1995	1
1996	2
1997	1
1998	2
1999	2

2000 rows × 1 columns

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: